Contents

lfp-efficient **1**
The problem . . . 1
Install . . . 1
Quick start . . . 2
The other question: show me the solutions . . . 2
And if you want all of them: the complete front . . . 3
The last gap: the complete efficient set, not just the front . . . 4
What an explicit `E(P_D)` buys: any decision function at all . . . 5
What is verified, and how . . . 7
Where the code reads differently from the paper . . . 7
Two additions the paper does not have, and one that did not earn its keep . . . 8
The hybrid that reaches all of `E(P_D)` — and the three that don't . . . 13
Solving the slice equations: the largest gain in the package . . . 15
A wall met four times . . . 15
How the numbers were measured . . . 16
Where this sits in the literature . . . 17
Layout . . . 17
License . . . 18

lfp-efficient

Exact optimization of a **linear fractional function over the integer efficient set** of a multi-objective program — pure Python, no dependency, rational arithmetic throughout.

Reference implementation of

> Leila Younsi-Abbaci, Optimizing a linear fractional function over an integer efficient set, **Reliability: Theory & Applications**, No 4 (40), Vol. 11, March 2025,

generalised to criteria that are themselves ratios (MOILFP).

The problem

```
(P_D)   "max"  Z_k(x) = (c_k'x + a_k) / (d_k'x + b_k) ,  k = 1..p
        s.t.   x in D = { x in Z^n_+ : A x <~ b }

(P_E)   max  Phi(x) = (U'x + alpha) / (V'x + beta)
        s.t. x in E(P_D)                    <- the *efficient* points of (P_D)
```

`E(P_D)` is a union of faces of `D`: non-convex, with no explicit description, so `(P_E)` is a global optimisation problem with many local optima in general. The method reaches the global optimum **without enumerating `E(P_D)`** — on the mid-size instances shipped here it gets there after generating 9–16 % of it. Linear criteria are the degenerate case `d_k = 0`, `b_k = 1`, so the paper's program is a special case and `MOILP` is a one-line constructor over `MOILFP`.

Install

```
pip install git+https://github.com/sisisoso2127-prog/hi.git
```

Python 3.10+. Nothing else is required, and that is a design property rather than an omission: the package carries its own exact simplex and branch & bound over `fractions.Fraction`, so every test — efficiency, a vanishing reduced gradient, an integral vertex — is an exact comparison with no tolerance anywhere. CI imports it on a bare interpreter to keep that true.

Quick start

```
from lfp_efficient import Model, MOILP, FractionalObjective, LE
from lfp_efficient import optimize_over_efficient_set

D = Model(2).add([-2, 1], LE, 0).add([6, 1], LE, 21).add([-2, 4], LE, 6)
problem = MOILP(D, criteria=[[1, -2], [-1, 4]])
phi = FractionalObjective(U=[1, 1], V=[5, 1], alpha=-1, beta=-1)

solution = optimize_over_efficient_set(problem, phi)
print(solution.x, solution.value)        # [3, 3]  5/17
print(solution.proved_optimal)           # True
```

The answer is a `Fraction`, exact. `solution` also carries a certified upper bound and the audit trail of every iteration; with `time_budget=...` the method becomes **anytime** — stop whenever and get a real solution attained at a known efficient point, a bound, and the absolute gap between them.

The other question: show me the solutions

The rest of this package answers which efficient point maximises *Phi*, and proves it. `efficient_subset` answers the question a decision maker usually asks instead — **show me efficient solutions with their `Phi`, so I can look at them**:

```
from lfp_efficient import efficient_subset

result = efficient_subset(problem, phi)
print(result.report())
for x, value in zip(result.points, result.values):
    print(x, value)
```

```
7 efficient solutions (every one certified)
  Phi* = 5/17 at (3, 3)  [proved optimal]
     7 from the Pareto archive
  this is a SUBSET of E(P_D); completeness is not claimed
```

Nothing here is new computation — it packages two sources the machinery already had:

source	coverage of `E(P_D)`	cost
the exact search's own trail	51 %	free — a by-product of proving optimality
the Pareto archive, verified	98–100 %	0.07 s walk, **0.25–0.62 s to certify**
augmented Tchebychev	39 %	0.60 s

The Tchebychev generator is not included, and that is a measured decision. On its own it reaches 39 %, and on top of the archive it adds **nothing at all** — every point it found was already there. It would cost roughly ten times the archive's walk and deliver no extra solution. It stays exported for anyone who wants it deliberately.

Certification is the cost, not the search. The archive filters by dominance among points it has seen, which is not the exact test — about one member in 2000 turns out not to be efficient. Certifying the whole archive is one integer program per member, six to eight times the walk itself. It is on by default, because an uncertified "efficient set" is a claim rather than a result; `certify=False` is there for anyone who knows what they are holding.

Two things it does not claim. It is not the whole efficient set, and the coverage above is measured against exhaustive enumeration on instances small enough to enumerate — where

`E(P_D)` cannot be enumerated there is no way to know what fraction was found. And the exact search's trail is a **biased** sample, not a representative one: it is exactly the points the search had to cut on, so it concentrates where `Phi` is large. For ranking by `Phi` that bias points the useful way, but it is a bias.

And if you want all of them: the complete front

`efficient_subset` never claims completeness, and that is a real limitation rather than a modest phrasing. `enumerate_front` removes it:

```
from lfp_efficient import enumerate_front

front = enumerate_front(problem, phi)
print(front.report())
```

```
7 non-dominated vectors -- the complete front, proved
  15 boxes settled
  best Phi on the front: 5/17 at (3, 3)
```

`enumerate_nondominated` already did this, but **in decision space** — Sylva–Crema cuts, `p` binaries and `p+1` big-M rows per cut, the model growing all the way. That is the cost profile this whole package exists to avoid, and the same substitution works on enumeration:

instance	|front|	criterion space	decision space	ratio
`n=4`	9	0.12 s	1.37 s	11.4×
`n=5`	**30**	**0.63 s**	**129.21 s**	**204×**
`n=5`	9	0.29 s	0.81 s	2.8×
`n=6`	6	0.08 s	0.63 s	8.0×

The ratio tracks the size of the front, not `n`. Every vector is a cut, and in decision space every cut adds `p` binaries and `p+1` big-M rows — so a 30-vector front leaves 90 binaries on the last sub-problem, while the criterion-space list never grows. The instance with the largest front is the one with the 204× gap; the two smallest fronts give 2.8× and 8.0×.

That is the same mechanism as the optimisation comparison further up, where the margin tracked the number of cuts rather than `n` — measured independently here, on a different task.

Four instances, and the spread is wide. The mechanism explains the spread, but these are four runs, not a distribution.

Is the baseline fair? A 204× margin invites the suspicion that the loser was handicapped, so we profiled it. Of the 137 s the `n=5` instance spends producing 30 vectors, **100% is inside the probe** — 31 calls, 14983 branch & bound nodes — against 0.09 s for the repair and 0.02 s for adding the cuts. The model grows from 5 columns and 7 rows to 95 and 127, exactly `p` binaries and `p+1` rows per cut. The cost is structural: it is the growth of the sub-problems, which is the thing the box search removes.

One candidate handicap remained: the probe only needs some feasible point, yet it maximises a direction. Stopping at the first integer point found looks strictly cheaper. It is the opposite:

probe	cuts	B&B nodes	time
`n=4` maximise	9	658	**1.35 s**
`n=4` zero objective	9	1397	5.33 s
`n=5` maximise	9	372	**0.79 s**
`n=5` zero objective	9	2105	10.15 s

The cut counts are identical — the direction changes neither how many vectors there are nor the order they appear in. What explodes is the branch & bound inside each probe, because a zero objective gives it nothing to prune with: every node's relaxation is worth 0, so no bound can discard a subtree until an integer point has been stumbled upon. The direction is load-bearing, the criterion-space method uses the same one, and the 204× stands.

Recorded because it is a plausible-looking optimisation that makes things 4–13× worse, and because it was checked while suspecting the comparison was unfair — it was not.

Why it is complete, and not merely large. A non-dominated vector `v` leaves the unexplored region only through a split around a centre `a` with `v ≤ Z(a)`. If `v ≠ Z(a)` that says `a` dominates `v` — which no non-dominated vector admits. So `v` can only leave at the moment it is recorded, and since the region is exhausted, every `v` is recorded exactly once. Termination is the same argument as the optimisation search's: the probe returns `x` inside the box and the efficient point it is repaired to dominates it, so the split removes at least `Z(x)`.

The `complete` flag is set only when the box list actually emptied. A run stopped by `max_boxes` or `time_budget` returns what it has with `complete = False` rather than a front it cannot support.

The front is not the set of efficient points. One vector can be attained by several efficient points, and the enumeration keeps one per vector. On random instances this never shows — 476 efficient points on 476 distinct vectors over 32 of them — but that is the sample, not the problem: generic coefficients make ties improbable. Built deliberately, `|E| = 6` sits on 2 vectors and the enumeration returns 2 of the 6 points.

That instance also shows what `Q` is for: its slice holds three points with `Phi = 3/4, 6/5, 3/2`; the repair lands on the first and `Q` moves it to the third. So the largest value on the front is still the optimum — and without `Q` the answer would have been **half** of it, with the front still correct and the output looking entirely reasonable.

Vectors are not solutions. `Phi` is a function of `x` and not of `Z(x)`, so one non-dominated vector can carry several efficient points with different `Phi` — enumerating the front does not by itself answer `(P_E)`. Given a `phi`, each vector is therefore paired with the point maximising `Phi` **on its slice**, which is what the plain vector enumeration cannot give you.

That pairing has a consequence worth stating: **the largest value on the front is the optimum of `(P_E)`**. So this is a second, independent way to solve the problem — and the suite uses it as a cross-check on the main algorithm.

What the pruning is worth, measured. Two exact routes now exist: prune (stop as soon as the rest cannot beat the incumbent) or enumerate (walk the whole front, take the best). They share the box machinery but not the reason they stop, so comparing them is meaningful:

	boxes	time
prune — `optimize_in_criterion_space`	102	**1.52 s**
enumerate — `enumerate_front`	215	2.96 s

Same optimum on all 8 instances. **1.94×** and half the sub-problems — and that is less than one might expect: enumerating the entire front costs only twice what finding the single best point costs.

The last gap: the complete efficient set, not just the front

`enumerate_front` proves its **vector** list complete, and says in the same breath that the **point** list is not — one point is kept per slice. That was the last honest gap, and it closes for almost nothing:

```
from lfp_efficient import complete_efficient_set
```

```
result = complete_efficient_set(problem, phi)
print(result.report())
```

```
6 efficient points on 2 non-dominated vectors -- the complete efficient set, proved
  5 boxes settled, variable box (2, 2, 2)
  largest slice 3 points, 2 of 2 vectors carry more than one
  best Phi over E(P_D): 3/2 at (2, 1, 2)
```

Why it is cheap. If `v` is non-dominated then **every** `x` with `Z(x) = v` is efficient — because a `y` dominating such an `x` would give `Z(y) ≥ Z(x) = v` with a strict inequality, making `v` dominated. So

```
E(P_D) = union over non-dominated v of { x in D : Z(x) = v }
```

and **not one efficiency test is paid on any slice point.** The most expensive operation in the package is simply not invoked.

A slice is `D` plus `p` equalities. Fixing `Z(x) = v` is linear in `x` even for fractional criteria: since `d'x + b > 0` on `D`, the equation `(c'x + a)/(d'x + b) = v_k` clears to `(c − v_k·d)'x = v_k·b − a`. No binary, no big-M, nothing that grows from one slice to the next — the same property the box search has, arrived at a second time.

What it recovers, and what it costs. Five variables enter `Z`; `f` further variables enter only their own bound `0 ≤ x_j ≤ 4`, so they are invisible to `Z` and every slice is a grid of `5^f` points:

f	|front|	|E(P_D)|	ratio	front	slices	overhead
0	7	7	1×	0.110 s	0.016 s	24%
0	13	13	1×	0.247 s	0.018 s	10%
1	7	35	5×	0.114 s	0.019 s	27%
1	13	65	5×	0.240 s	0.024 s	13%
2	7	175	25×	0.117 s	0.036 s	42%
2	13	325	25×	0.246 s	0.050 s	24%
3	**7**	**875**	**125×**	0.128 s	0.103 s	90%
3	**13**	**1625**	**125×**	0.259 s	0.167 s	68%

At `f = 3` the front is **complete and missing 99.2% of the points** — exactly the distinction the section above insisted on — and recovering all 1625 of them **never doubles the cost**. The scan is output-sensitive: 0.1 ms per point, against 7.68 ms for one efficiency test, and there are no tests to pay.

The honest reading, which is conditional. At `f = 0` the front already was the efficient set, and the 10–24% buys only the proof of that — which is the usual case on generic random instances (476 points on 476 vectors over 32 of them). This is not a speed-up and it is not uniform. What it buys is that the method now **knows** which case it is in, instead of returning a point list whose completeness rested on an assumption about the coefficients. Where ties do occur, the front alone silently returns a fraction of the answer.

What an explicit `E(P_D)` buys: any decision function at all

This is the part worth reading if you read only one section, because it does not rest on being fastest at anything — it is about which problems can be posed.

> A decision-space method needs **structure in `Phi`** and nothing of `E(P_D)`; this route needs **`E(P_D)`** and nothing of `Phi`.

A branch-and-cut for a linear fractional `Phi` builds its cuts, bounds and Dinkelbach steps out of the ratio form. Replace `Phi` by a sum of two ratios and it does not slow down — it stops applying. Over a finite, explicit, certified `E(P_D)` that case costs exactly what one ratio costs, because the optimum of a function over a finite explicit set is settled by evaluation.

```
from lfp_efficient import (complete_efficient_set,
                           maximize_over_efficient_set,
                           nondominated_over_efficient_set)

result = complete_efficient_set(problem)        # proved complete, or it refuses

maximize_over_efficient_set(result, lambda x: phi1(x) + phi2(x))   # sum
maximize_over_efficient_set(result, lambda x: max(phi1(x), phi2(x)))
maximize_over_efficient_set(result, f, minimize=True)
nondominated_over_efficient_set(result, [phi1, phi2])   # several at once
```

Six shapes on the published instance, each checked against brute force:

decision function	admitted by decision-space branch-and-cut?	optimum
`Phi1`	yes — the classical case	5/17
`Phi1 + Phi2`	**no**	47/35
`max(Phi1, Phi2)`	**no**	6/5
`Phi1 * Phi2`	**no**	6/35
`(x1^2+1)/(x2^2+1)`	**no**	10
piecewise / black box	**no**	3/11

Two linear fractional functions at once — its own problem in the literature, with its own branch and bound and cutting planes — is a dominance filter here, and the cost depends neither on how many functions are asked for nor on what they are. On the published instance it keeps 4 of the 7 efficient points, exactly the brute-force answer. Ties are reported rather than resolved arbitrarily, which matters because `Phi` is a function of `x` and not of `Z(x)`.

Both entry points **refuse a `CompleteSet` that is not proved complete**. A budget-truncated set still holds genuine efficient points, so a maximum over it is still a number — just not the answer to the question asked, and nothing downstream could tell the difference.

The price, measured, and it is not always worth paying. Against running the exact method once per decision function, building `E(P_D)` first pays after a median of **2.4** decision functions — IQR [1.8, 2.9] over 117 instances, extremes 1.2 and 7.2, thirty instances per configuration.

The statistic is in **sub-programs, not seconds**, which is what lets it be quoted that tightly: a scan over an explicit `E(P_D)` solves no integer program, so the break-even is exactly `programs to build / programs for one exact run` — deterministic, identical on every machine and every repeat, and untouched by the ±30% noise floor. Three of 120 runs were censored when an exact run spent its budget; that censoring is one-sided against this method (an overrunning exact run is an expensive one, which is where building `E(P_D)` pays soonest), so 2.4 is an upper bound on the truth.

> **A withdrawn figure.** An earlier version of this section reported a median of **13**, range 1 to 2117. That is withdrawn. It came from eight hand-made instances with a deliberately conflicting criterion structure and `|E(P_D)|` between 17 and 105; the campaign family has `|E(P_D)|` between 4 and 9, and a small efficient set is cheap to build. The old number is what those eight instances did — but it was quoted as the break-even when it was the break-even of one family, which is the error this project records making five times before. This was the sixth, caught by re-measuring.

For a single linear fractional `Phi` the decision-space method is often the better buy: it never builds `E(P_D)` and is untouched by its size, while this route builds it and is bounded by it. They fail on different instances. The break-even is 1 only for an `f` outside the linear fractional class, where there is no competitor to amortise against — and that part does not depend on the measurement.

What is verified, and how

`python tests/test_lfp_efficient.py`	106 tests, no pytest needed (it runs under pytest too)
`python examples/paper_example.py`	reproduces §4 of the paper: `X_opt = (3,3)`, `Phi_opt = 5/17`
`python examples/large_example.py`	instances up to |D| = 34635, each cross-checked against an independent scan
`python examples/fractional_example.py`	fully fractional criteria, checked against Definition 1 point by point
`python examples/scaling.py`	certified optimal up to `n = 20` (slow; opt-in in CI)

Correctness rests on **four independent references**, not on the algorithm agreeing with itself: exhaustive enumeration, a repeated-cut enumeration of the non-dominated set, a sorted scan with dominance tests, and `certify_optimum`, which settles 362293 challengers with 3 exact efficiency tests.

And the references themselves were audited. Eleven places were found where a hypothesis stated in the documentation was never checked in the code — all fixed, all carrying regression tests, and no measured result moved. The pattern is worth knowing if you build on this package:

- Every flag claiming something was proved was computed from control flow ("the loop finished"), not from the proof's hypotheses. "The box list is empty" was reported as "the front is complete", which holds exactly when `D` is bounded and the denominators are positive — neither of which was established.
- **Every parameter added to save work was a way to supply a hypothesis without proving it.** A precomputed variable box, a precomputed front, seeds, an incumbent and its value: each was introduced as an optimisation and quietly became a trust assumption. A caller passing a variable box asserts it contains `D`, and nothing downstream can detect one that does not — every routine receiving it searches it honestly and reports a complete-looking answer about a feasible set that is not `D`.
- The two worst sat in the verification apparatus itself: `certify_optimum` certified a suboptimal point when handed a clipped box (`[1,1]` certified `Phi = 1/9` while the optimum is `5/17`), and both reference oracles did the same — answering about the efficient set of a different feasible set under the names `E(P_D)` and "the exact optimum". A clipped oracle does not give a wrong answer; it gives a wrong standard.
- None of it was visible to the suite because all 93 tests then in it fed **valid** input. They established the method is correct when its hypotheses hold — never the question — and could not establish that anything enforced them. The ten added tests all feed violating input and require a refusal.

The mathematics survived unchanged: the slice characterisation, front completeness, split disjointness and big-M validity were re-derived and re-tested, the last on 220 point pairs with no violation and the bound exactly attained. All eleven findings were in the enforcement of hypotheses, never in the reasoning that uses them.

Where the code reads differently from the paper

The long-form notes live in **`lfp_efficient/README.md`**: the full method, the performance work (two orders of magnitude on `n = 4`), the scaling study, and — stated rather than papered over — every place the implementation departs from the printed text. The short version:

- **The printed "Algorithm 2: part 2" is internally inconsistent.** It stores `X_opt = x_l`, the point just shown not to be efficient, while setting `Phi_opt = Phi(x~_l)`. The implementation keeps the mathematics and fixes the bookkeeping.

- **The paper's own numerical example violates the standing assumption of linear fractional programming** that `V'x + beta > 0` on the feasible set: `x = (0,0)` is in `D` and gives `-1`. The solver splits the feasible set along the sign of the denominator and solves both branches.
- **Ecker & Kouada does not transfer to ratios.** With linear criteria the maximiser of the efficiency test is itself efficient; with fractional ones it only dominates, because the k-th term carries a factor `D_k(x)` that varies from point to point. The dominance chain is walked, and a test measures that every linear chain is one step while a share of fractional ones are longer.
- **The edge step of Definition 2 is nearly inert**, measured rather than assumed: of 174 zero-gradient columns over 80 instances only 8 move `x` at all, and the step fires on about one instance in eighty. It is implemented, repaired and kept — but the README says plainly that it does not make the method faster.

Two additions the paper does not have, and one that did not earn its keep

A criterion-space search (`optimize_in_criterion_space`). The paper cuts in decision space, where "delete `{x : Z(x) <= Z(x~)}`" is a disjunction needing `p` binaries and `p+1` big-M rows per cut — so the region grows every round and the late sub-problems dominate the cost. Keeping the remainder as a list of boxes in criterion space instead makes every sub-problem `D` plus a few ordinary linear rows, and the model never grows:

	decision space	criterion space	
35 instances, total	40.65 s	**8.06 s**	**5.05×**
heaviest `n=10`	23.47 s	**2.61 s**	**9.01×**

Same optimum everywhere, both proved optimal, faster on 34 of 35 — and the margin widens with difficulty. It **widens with the number of criteria** too, which is where the gap becomes a different order of magnitude: at `p = 5` on six variables the paper's method took 110.95 s against **0.64 s**.

Thirty instances per row, `n = 6`. The statistic is the median of the per-instance ratios — a ratio of totals is decided by the single heaviest instance:

criteria	decision	criterion	ratio (median)	IQR	max
`p = 2`	0.07 s	0.04 s	1.52×	[1.25, 1.91]	6.0×
`p = 3`	0.34 s	0.12 s	2.69×	[1.87, 3.72]	10.1×
`p = 4`	0.50 s	0.13 s	4.99×	[2.39, 11.62]	76.3×
`p = 5`	2.19 s	0.25 s	7.93×	[3.61, 19.98]	33.8×
`p = 6`	3.67 s	0.31 s	**14.57×**	[7.90, 25.17]	65.7×

Three of the 150 runs hit a 45 s budget (one at `p=5`, two at `p=6`) and are excluded rather than counted as their budget. That censoring is **one-sided** — it can only flatter the decision-space method — so these margins are conservative. Every instance that finished agreed on the optimum.

An earlier table here said the curve turned over. It does not. Four instances per `p` gave 1.74, 2.99, 6.92, **103** and 24.3, and the text went on to explain the turn between `p=5` and `p=6`. There was nothing to explain: that 103× was one instance supplying 110.95 of 112.77 seconds. With thirty instances the progression is monotone. The old numbers were not wrong as measurements — they are what those four instances did — but they were quoted as a trend, and four runs cannot carry one.

Where the margin comes from, counted. Wall-clock cannot resolve a small effect on this machine, so the campaign counts deterministic work too:

p	sub-programs	b&b nodes	time
2	1.72×	1.28×	1.52×
3	1.40×	1.79×	2.69×
4	1.19×	2.68×	4.99×
5	1.17×	3.16×	7.93×
6	**1.02×**	**5.37×**	**14.57×**

At `p = 6` the two methods solve **the same number of sub-programs** — 1.02×, down steadily from 1.72× at `p = 2` — and one still takes 14.57× as long. The entire margin has moved into the cost of one sub-problem: the branch & bound inside it explores 5.37× the nodes, because by then it carries `p` binaries and `p+1` big-M rows per cut while the box search carries none. That is the cost model read straight off the measurement.

It supports fractional criteria too, and keeps the certified gap and `time_budget`. Both methods stay: this package is a reference implementation of the paper, and this search is not in the paper.

A hybrid (`optimize_hybrid`) runs the paper's loop and, if it has not closed after `switch_after` iterations, replays its cuts as a box list and carries its incumbent over — so nothing proved is proved twice. Measured on 24 instances against the better of the two pure methods on each:

	total	within 15% of the better
`switch_after = 3`	13.14 s	4 of 24
`switch_after = 1`	8.47 s	23 of 24
pure box search	**8.11 s**	—
the paper's method	268.61 s	—

Read that honestly: at `switch_after = 1` the hybrid **ties** the pure box search (8.47 s against 8.11 s, and 1.47 s against 1.54 s on a family where the paper's method closes in one iteration — two ~4% differences pointing opposite ways). It is insurance for the one-iteration case, not a third method that beats both. `switch_after = 3`, the first default, was measurably wrong.

An exact–metaheuristic hybrid (`optimize_hybrid_metaheuristic`) runs a Pareto local search first and hands the box search a **verified efficient** incumbent. 9.50 s → 7.60 s over 18 instances (1.25×), reaching 1.56× on the heaviest and losing only where the exact search already took milliseconds. The ceiling, measured by handing over the true optimum for free, is 1.42×.

On thirty paired instances the win is smaller, and one verdict softens. Each instance at `n=6, p=4` is solved three times; the statistic is the median of the per-instance speed-ups:

seed	speed-up (median)	IQR	seed gap	exactly optimal
Pareto archive	1.10×	[1.02, 1.27]	0.000	24/30
Tchebychev	1.02×	[1.00, 1.09]	0.170	8/30

The **ordering** is what every sample has said and is the part that holds. Two things change. The Pareto seed is worth less than 1.25× here — median 1.10×, first quartile 1.02×, so half the instances gain three percent or less; the win is real (both quartiles above 1, exactly optimal on 24 of 30) but small, and the worst instance runs at **0.618×**, a genuine loss, because verification is paid whether or not the bound can use it. And the Tchebychev seed **does not lose here; it does nothing** — 1.02×, IQR [1.00, 1.09], a wash rather than the 0.71× reported below. The verdict that survives both samples is the weaker one: it never pays. What is stable is the quality figure that explains it — a median gap of 0.170 against 0.000, and 8 of 30 exactly optimal against 24 of 30.

The contrast is the interesting part: the same free optimum saves **zero iterations** in the paper's method, because what it cuts is decided by the efficiency test and not by the incumbent. The worth of a heuristic incumbent depends on which exact method it is hybridised with.

An early exit on a hopeless best bound. The box loop pops from a heap ordered by inherited bound descending, and used to solve every box it popped. But once the best bound still open fails to beat the incumbent, so does every other, and the whole remaining tail is busywork. Measured before changing anything: 198 of 733 solves (27 %) had such a bound — and on **every one of the 18 instances** that count equalled the number of boxes still open when the condition first fired. The waste is exactly a tail, never scattered, which is what the heap order predicts. Stopping there takes **733 boxes to 549, 25 % fewer**, same answers, still proved optimal.

It is **not a speed-up**: 7.44 s → 7.27 s at per-instance minima over five repeats, with per-instance ratios scattered from 0.77× to 1.36×. Since the change can only remove work, anything under 1.00× is machine noise — and the noise is bigger than the effect. That is a confirmation rather than a disappointment: the tail boxes are the cheapest ones, each killed by the root relaxation the moment it sees the cutoff. A quarter of the sub-problems were genuinely wasted and worth almost nothing.

And the same measurement settles a question worth asking before stacking ideas: on the metaheuristic-seeded hybrid this exit cuts only **4 %** of boxes (562 → 539), against 25 % unseeded. The seed and the exit are **substitutes, not complements** — a good incumbent arrives early enough that the hopeless tail barely forms.

Cutting on a point the search already proved efficient. Splitting a profile of the work by operation put the cost somewhere unexpected: the efficiency test is **52 % of all simplex pivots on 146 calls** — 58 pivots each against 11.8 for a box solve. So the question stopped being "make the test cheaper" and became "pay it less often".

When a box's maximiser `x_b` is inefficient the search runs that test to find an efficient point dominating it. But it already holds a list of points it has proved efficient, and finding one that dominates `x_b` is arithmetic rather than an integer program. Measured before building: one does on **21 of 146 tests (14 %), and on 17 % of the inefficient cases**.

	base	shortcut
simplex pivots	16169	**14602** (−9.7 %)
boxes solved	549	**534** (−15)
optima	—	identical, all proved

The boxes went down, which was not the prediction. Among the remembered points that dominate `x_b` the highest is taken, and `{Z ≤ Z(centre)}` removes more of the box the higher its centre — so skipping the test and cutting better compound instead of trading off. One instance gained a box; the rest lost or held.

Soundness rests on one invariant: **every point the search remembers is efficient.** `explored` holds the centres it cut on and the `Q` maximisers beside them, and a `Q` maximiser shares its centre's criterion vector, so it is efficient whenever the centre is. A single inefficient member would let the search cut on a dominated point and delete the true optimum, so the invariant is a test rather than an argument — 0 failures over 34 remembered points, and the shortcut itself is checked against the independent scan rather than against a box count, since it deliberately changes which boxes are produced.

Warm-starting a box from its parent. A box's region is its parent's plus a handful of rows, yet every box solved its root relaxation from scratch — and `solve_standard_form` needs a phase I whenever a row is not covered by a slack, which every `e_k ≥ 1` row forces. Measured first, before anything was built: **4992 of 18193 simplex pivots (27 %) were phase I inside cold roots**, and 788 of 824 roots (96 %) paid one. Phase I costs 6.3 pivots per root against 2.3 for phase II —

the same “five sixths thrown away” the package already measured inside a branch-and-bound tree, one level up.

`add_linear_row` is the general form of the branch and bound’s own `add_bound_row`: it appends `coeffs·x ≤ rhs` to a solved tableau and eliminates every basic variable in it, which one pass does because a basic variable’s column is a unit vector. The parent’s basis then stays a basis once the new slacks join it, only the new rows can be primal infeasible, and the existing dual restoration fixes that.

	base	warm
simplex pivots	18193	**16169** (−11.1 %)
cold root relaxations	824	**307** (−63 %)
boxes, MILP programs, optima	—	identical

The ceiling was never 27 %. Only a box with a parent can inherit one, and that is 63 % of roots — the rest are the 146 efficiency tests and 125 `Q` solves, which extend `D` rather than another box, and `D`’s own basis is free (all its rows are `≤` with non-negative right-hand sides, so the crash basis covers them and phase I is skipped outright). So the reachable ceiling was ~17 %, and 11.1 % of it survives after the restoration’s own 1583 pivots.

517 warm starts, 0 stalls — the cold fallback never fired once. It exists anyway, because a restoration that gives up must cost time and never correctness.

The gain grows with `p` (28 % at `p=7`, 1–7 % at `p=3`), which is what more rows per box predicts, and two tiny instances lose a little where the restoration costs more than the phase I it replaces.

Not rediscovering Ecker & Kouada once per box. When a box’s maximiser fails the efficiency test, the search needs an efficient point to cut on instead, and called `repair_to_efficient` on the test’s witness. That walk begins with a full efficiency test on the witness — and with linear criteria the answer is known in advance. The test maximises $\Sigma_k(Z_k(y) - Z_k(x^*))$, a strictly positive weighted sum of `Z` up to a constant, over `{y ∈ D : Z(y) ≥ Z(x*)}`. If some `z` dominated a maximiser `y*`, then `Z(z) ≥ Z(y*) ≥ Z(x*)`, so `z` lies in that same region and scores strictly higher — contradiction. No maximiser is dominated, ties included.

Checked before being relied on: **349 witnesses of failed tests across 60 linear-criteria instances, none of them dominated.** `efficient_dominator` takes the shortcut when the criteria are linear and falls back to the walk when they are not — with a ratio the $D_k(y)$ factor breaks monotonicity in `Z` and a dominating point can score lower.

	base	shortcut
integer programs	1466	**1341** (−8.5 %)
branch-and-bound nodes	8105	**7618** (−6.0 %)
boxes solved, optima	—	identical

A note on how that was measured, which matters more than the 6 %. The wall-clock A/B for this change reads 0.96× — slower — and is faster on 9 of 18 instances. That cannot be causal: removing 125 integer programs cannot cost time. This machine’s noise floor is roughly ±30 % per instance even at per-instance minima over five repeats, so it simply cannot resolve a 6 % effect. The table above therefore counts **deterministic work** — programs and nodes — which is the same number on every run and every machine.

The prediction was wrong too, and in an instructive way. The profile that motivated the change showed the efficiency test at 18.4 ms/call, the dearest operation in the search; but what the shortcut removes is the repair’s test at 7.68 ms — the cheap kind, because proving `θ = 0` on a point already known efficient is easier than searching for a dominator. 8.5 % of the programs are only 6.0 % of the nodes. Reasoning from the mean cost of a call would have overstated this by half.

The shortcut is used at all four sites that pass a failed test's witness, so the paper's own method gets the same saving and the comparison between the two stays fair.

The augmented weighted Tchebychev program (`augmented_tchebychev_efficient`) is the scalarisation the surrounding literature reaches for — Chaabane, Brahmi and Ramdani (2012) optimise over an integer efficient set with it, Younsi-Abbaci and Moulai (2021) use it over the Pareto front. It is implemented here as a **generator of certified efficient points**, and measured against the two generators already in the package.

It is strictly stronger than the weighted sum, which is the reason the literature prefers it. A weighted sum can only maximise at a supported efficient point; varying the Tchebychev weights reaches unsupported ones too. Over 20 random instances, from the same grid of 27 weight vectors:

	efficient points reached	of which unsupported
weighted sum	66 / 175	**0 / 54**
augmented Tchebychev	112 / 175	29 / 54

and every one of the 540 optima it returned was efficient, none merely weakly so — which is what the augmentation `rho` buys. Those are the numbers `examples/tchebychev_study.py` prints; a wider sweep over 40 instances and 1080 programs gives the same picture (216/369 against 135/369, 56 unsupported against 0, and again no inefficient optimum).

As a seed for the box search it never pays, and the table says why. Against the same 18 instances used for the metaheuristic above — where it reads as a loss; on thirty paired instances it reads as a wash (1.02×), and never pays is the verdict that survives both:

seed	total	seed cost	exactly optimal
none (pure box search)	9.64 s	—	—
Pareto local search	**7.97 s** (1.21×)	0.57 s	12 of 18
augmented Tchebychev	13.50 s (0.71×)	4.25 s	5 of 18

Both halves of the loss are structural, not an artefact of this implementation. The cost is `p + 1` integer programs, each carrying an extra continuous variable and `p` extra rows; the ideal point is not the culprit (0.01–0.13 s of the seed, against 0.13–0.85 s for the scalarisations themselves). And the quality gap is the point: the Tchebychev program steers by criterion-space geometry relative to `z*` — **it never looks at `Phi`**. The local search is guided by `Phi` at every move. A generator of a good spread of efficient points is not the same thing as a generator of good `Phi`.

That ordering is stable but the size of the win is not, and the two should not be confused. On the looser family in `examples/tchebychev_study.py` the Pareto seed is exactly optimal on 15 of 24 instances and still buys only 1.05×, because that search is not bound-limited — seed **quality** is not seed **value**, which is the same lesson as the zero-iteration result above. Tchebychev loses on every family tried (0.73×, 0.91×); the Pareto seed ties or wins on every one.

The same split shows up when the two are compared as front generators rather than as seeds. On 20 instances at `n = 4`, Tchebychev reached 64% of `E(P_D)` in 2.1 s and the Pareto archive 99% in 0.04 s. But the archive filters by dominance among the points it has seen, which is not the exact test: over a wider sweep one member in roughly 2000 turned out not to be efficient, while every Tchebychev optimum is efficient by construction. Rare is not never, and that is precisely why `metaheuristic_incumbent` puts the archive's best through `test_efficiency` before any bound is allowed to cross into the exact search.

A note on how this was measured: an earlier version of the harness scored a seed by the ratio `Phi(seed) / Phi*`, which **inverts when `Phi*` is negative**, as it is on some of these instances. The tables above use the non-negative gap `(Phi* - Phi(seed)) / |Phi*|`, zero exactly when the seed is optimal.

Batch cutting (`batch_cuts_after`) cuts on several cheaply generated efficient points at once, taking the heaviest instance from 11 step-1 solves to 8 (27.1 s → 15.7 s). It is **off by default**, because on instances that were never hard it only grows the model — the notes carry the losses alongside the win.

The hybrid that reaches all of `E(P_D)` — and the three that don't

Everything above reaches `E(P_D)` by a **purely exact** method. Every hybridisation so far was hung on the optimisation search, where a seed feeds the **bound**. The front enumeration has no bound and no incumbent — it must exhaust every box — so a seed of that kind is worth exactly nothing there.

It has a different mechanism: every box costs a probe, an efficiency test and sometimes a repair, all to find a vector, and a vector already known needs none of them. Seeding here is not "start from a better value" but **"don't pay to discover what you already know"**.

```
from lfp_efficient import hybrid_complete_set

result = hybrid_complete_set(problem, phi)
print(result.report())        # same E(P_D), same optimum, ~1.31x faster
```

	median	IQR	faster on
front enumeration	1.33×	[1.20, 1.51]	139/150
`E(P_D)` complete	**1.31×**	[1.17, 1.47]	139/150
probes	0.72×	[0.67, 0.75]	never worse

Thirty instances at each of five configurations. **Identical answers on all 150** — same front, same `E(P_D)`, same optimum — asserted, not assumed. The heuristic decides only how much of the front has to be discovered.

Why the walk pays where the generator didn't. A Tchebychev scalarisation costs 20.90 ms and buys one point; a Pareto walk costs 0.011 s and buys a whole archive covering **95% of the front**. The walk's points need verification and Tchebychev's don't — and that verification is **not a cost here**, because the enumeration was going to pay the same efficiency test on every probe the seed removes. It cancels; the probe is the gain.

Seeding from the generator instead is worse the more of the front it seeds: with a wide weight grid it covers 100% and runs at **2.61×** the unseeded time. One probe is 9.48 ms and 0 b&b nodes; one scalarisation is 20.90 ms and 21 nodes, cheaper on only 7 of 40.

And a 1.33× win was sitting behind a sort. In archive order the hybrid lost at 0.81×, with the probe count rising on 4 of 12 instances — the pre-split was building a box structure the search would not have chosen:

seed order	probes	raised the count on
archive order	0.89×	4 of 12
probe order	**0.70×**	**none**
reversed	0.94×	5 of 12

The reversed arm is what makes the direction causal rather than a coincidence.

On fractional criteria — where the problem actually lives. Everything above uses linear criteria, which is a gap worth naming in a package about ratios. It has a reason (the Tchebychev comparison needs them), but the Pareto walk doesn't:

	|F|	cover	ratio
n=5 p=2	7	100%	1.54×
n=6 p=3	22	100%	1.35×
n=7 p=3	36	100%	1.25×
all 40			**1.39×**, faster on 34

At least as good as the linear case. Nothing in the construction distinguishes them — the e_k rows are integer-valued for ratios too.

Five instance families, and a prediction that was wrong. Everything else here comes from one generator. Four more move what the mechanism depends on — loose raises |D| an order of magnitude while leaving |F| alone; knapsack is two capacity rows, all coefficients positive; conflicting gives the criteria opposing signs (large |F|); aligned correlates them (small):

family	|F|	cover	ratio	faster on
mixed	8 / 30	100 / 97%	**1.37** / 1.30×	28/30
loose	9 / 44	83 / 86%	1.29 / 1.23×	28/30
knapsack	12 / 41	100 / 90%	1.33 / 1.27×	29/30
conflicting	107 / 105	100 / 93%	1.14 / 1.14×	26/30
aligned	6 / 11	100 / 84%	1.31 / 1.14×	25/30
all 150			**1.23×**	**136**

Every family pays, so the advantage isn't an artefact of the generator it was found with. **But it refutes a prediction we wrote down first:** we expected conflicting (large front, much to discover) to be the best case and aligned the worst. The opposite holds, and the margin narrows with |F| inside every family too.

What bounds it, and what doesn't. A seed removes one probe; the enumeration spends 2.3–7.5 probes per vector, the rest on boxes that turn out empty and that no seed can touch. So the **probe-count** saving is capped at one over that figure, and shrinks as it grows — sweeping p at n = 7:

p	2	3	4	5	6	7
probes / vector	2.3	3.0	4.0	4.9	6.4	7.5
probes kept	—	0.68	0.75	0.82	0.87	—
time	1.38×	1.48×	1.33×	1.21×	1.24×	1.10×

32% of probes removed at p=3, 13% at p=6; 72 instances, faster on 68. **That bound does not bound the time**: a seed removes a probe on a non-empty box, which carries an efficiency test behind it, while the probes left behind are mostly on empty boxes and are refuted at once. At p=6 the time ratio exceeds 1/(probes kept). An earlier version here called this "a hard ceiling of about 1.4×" — **withdrawn**. What grows with |F| is the walk's cost, one efficiency test per archive member: a large front buys no more and costs more.

Where it fails, and the one line that fixes it. At n = 12 the margin falls to 1.00×, and at n=12, p=5 to 0.97×. We first blamed the probe count outgrowing |F|. **The measurement refutes that** — probes per vector are flat across the range (2.4, 2.7, 2.7, 2.8, 2.8, 3.0, 3.9, 5.2). What collapses is the share of the front the walk finds, and the speed-up tracks it line for line:

	cover	probes saved	ratio
n=6	87%	29%	

	cover	probes saved	ratio
n=10	65%	24%	1.19×
n=12 p=3	35%	9%	1.00×
n=12 p=5	12%	2%	0.97×

A fixed budget of 8 restarts and 4000 neighbours is fixed work against a growing target — and 4000 was chosen at n=7,8 and generalised. Widening the walk alone takes n=12 from 35% coverage and 1.03× to **95% and 1.61×**, and **restarts** are the knob, not neighbours. So the first round is sized to the instance, at [n/4] times the restarts. Over **120 instances** spanning n = 6…12: median **1.36×**, faster on **116**.

But coverage is not the objective. On the p=5 instance, **46% coverage gives 1.20× and 93% gives 1.12×** — buying the last half of the front costs more walking than the probes it removes. A walk that restarts in doubling rounds until one finds nothing new takes coverage there from 45% to 97% and is slower: 1.02× against 1.14×, and a median 1.32× against 1.36× over the same 120, winning 109 against 116. It's in the code, switched off.

A correction to our own reading: the collapse was first reported as "35% coverage, 1.00×" from **one** instance; over six seeds of that configuration the unmodified walk averages 78% and 1.35×. Real and severe where |F| is large (45% at |F| = 302) — but not as general as one instance made it look.

Solving the slice equations: the largest gain in the package

Completing the front to E(P_D) was dominated by the slice scan — 25–30 s against under a second for the front at n ≥ 9, and at n = 12 a slice could not finish inside two minutes.

The p slice equations are **linear**, so solve them. Gaussian elimination in exact rationals expresses r variables as functions of the rest; substituting them into every model row leaves an equivalent system in n − r unknowns. The search space (ub+1)^n becomes (ub+1)^(n−r).

instance	prune	eliminate	ratio
n=9, p=3	29.71 s	0.31 s	96.7×
n=10, p=5	119.47 s	0.92 s	129.9×
n=12, p=3	**796.84 s**	**8.34 s**	95.5×
median of all fourteen			**49.2×**

Exactness is untouched — every coefficient is a Fraction — so this changes the order of the search and nothing else.

A sign error that ran, and returned nothing. The first draft put a plus where the eliminated variables' contribution must be **subtracted**. It ran without error, looked like a working optimisation, and returned the **empty set on 179 of 185 slices** — silently dropping efficient points. Only the comparison against the un-eliminated scan caught it, which is what that comparison is for.

A wall met four times

Four separate changes remove a fifth to a quarter of the sub-problems and buy nothing:

change	sub-problems removed	cost
early exit on a hopeless bound	25%	no measurable time
free contradiction filter	24.8%	0.98× pivots
range filter	+11.3%	1.03× pivots

change	sub-problems removed	cost
dominated-whole filter	8–23%	1.077× pivots

> **In this search, removing sub-problems does not remove time, because the ones that can be removed cheaply are the ones that were already cheap.**

An empty box — and **73% of them are** — is refuted by the simplex almost at once: every probe in these families costs **zero** branch & bound nodes, because the relaxation is integral or infeasible at the root. The only change that ever went round this wall attacked an **exponent** rather than a count, and is the 49× above. We state the wall plainly so the fifth attempt isn't worth anyone's afternoon.

The same wall on the proof, not just the search. After the walk seeds 95% of the front, 93% of the remaining boxes are empty and the enumeration is almost purely a proof of emptiness. "Is there a feasible point not dominated by any of the `k` known ones?" is **one** question, and Sylva–Crema answers it in one integer program built once — no iteration, no growth. It's sound: 13 of 30 instances proved complete by a single program, no wrong verdict. It is also **4.5× slower** in the median and 100× at worst (239 s against 1.95 s), because `k` cuts mean `k·p` binaries in one model.

That closes a wider question. Criterion space was claimed the better search because the model never grows; this tests whether it's also the better proof, where nothing grows at all. **It is, by 4.5×** — answering 1363 cheap questions beats answering one with 88 binaries.

How the numbers were measured

What the baseline is. The decision-space side of every comparison above is **this package's own reimplementation** of the published method (`lfp_efficient/algorithm.py`), on instances generated here — not the authors' code, and not their published benchmarks. Both sides share the same simplex, the same exact rational arithmetic, the same branch and bound and the same machine, and the reimplementation follows the printed algorithm step for step, with the two places where that pseudo-code is internally inconsistent corrected and the corrections documented in the module.

So the comparison **isolates the algorithmic difference**, which is a cleaner question than code against code — that would also measure language, solver backend and engineering effort. The price is that **absolute seconds carry no information outside this implementation**: pure Python over exact rationals is far slower than any production solver, equally on both sides, so only ratios transfer and only ratios are quoted. A comparison against the authors' own implementations on their own benchmarks has **not** been done and is not claimed; until it is, every margin here reads "this algorithm beats that algorithm when both are implemented the same way", not "this work beats those papers".

Most figures above were first taken on four to eight instances. The direction of each was argued from a mechanism, which is what made them worth reporting — but a total over four runs is not an estimate. `studies/campaign.py` re-measures five of the claims over **thirty instances per configuration** — the first four of them in 751 seconds — and writes every per-instance record to `studies/campaign-results.json` so the tables can be rebuilt and re-checked without paying for the run again.

```
python studies/campaign.py --quick    # a pilot
python studies/campaign.py            # the full campaign
```

- **The statistic is the median of the per-instance ratios**, with the IQR — never a ratio of totals. A ratio of totals is decided by the single heaviest instance, which is exactly how the 103× at `p = 5` came about.

- **Deterministic work is counted next to the seconds**: sub-programs and branch & bound nodes, identical on every run and every machine. This machine's noise floor is ±30% per instance, so seconds alone cannot resolve a small effect and no number of repeats fixes that.
- **Censoring is reported, never hidden.** A run that spends its budget is excluded from the medians and counted in the open; a row with more than half its runs censored says so instead of quoting a number.
- **Agreement is asserted, not assumed** — every experiment checks the two methods return the same answer, not merely that one is faster.

Nothing else ran on the machine during the campaign. A contaminated measurement is discarded rather than quoted — a rule adopted here only after breaking it once.

Where this sits in the literature

Two parts of this construction are **prior art and are not claimed**:

- **Criterion-space decomposition** for optimising over an efficient set — Boland, Charkhgard & Savelsbergh (EJOR, 2017) work in the full-dimensional criterion space and decompose it each iteration, which is structurally the box search here with a linear decision function; their balanced box, boxed line and triangle splitting methods are the bi-objective precursors.
- **Metaheuristic–exact hybridisation**, which has a known taxonomy in which "run a metaheuristic to approximate the Pareto set, then let an exact method refine it" is exactly the scheme used here.

What the fractional literature does not appear to do is work in criterion space. Optimising a linear fractional function over an integer efficient set is a sustained line — Drici, Ouail & Moulaï (Annals of OR, 2018), a 2015 RAIRO-OR paper, Ouail & Chergui (arXiv:1907.02036), Chaiblaine, Moulaï & Cherfaoui (arXiv:2003.05364), a 2021 Optimization paper on quadratic objectives, and the stochastic and interval variants of Younsi-Abbaci & Moulaï (2021–2023) — and all of them search decision space with branch and bound or cutting planes.

So what is plausibly unaddressed is narrower than "the method": the **combination**, together with what makes the transfer exact rather than approximate (the integer-valued `e_k`, which replaces an estimated minimal step by `e_k >= 1` exactly, and the slice results), and what that combination then buys — the admissible class above.

This is a statement about what was and was not found, not a claim of novelty. No full text in that list has been read; it was assembled from abstracts and listings. One paper bears directly on the complete-set result and must be read before it is called new: Generating the efficient set of multi-objective integer linear plus linear fractional programming problems, Annals of Operations Research **296** (2021), pp. 735–753.

Layout

Path	Contents
`lfp_efficient/`	the package — and its long-form notes in `README.md`
`examples/`	the paper's example, larger instances, the scaling study, the fractional case, the complete efficient set
`tests/`	the test suite, runnable with a bare interpreter
`studies/`	the measured campaign, and earlier metaheuristic experiments kept for the record
`docs/`	the write-up: every method, every measurement, and every negative result

License

MIT — see `LICENSE`.